

Reviewer Pack — Minimal Rank of p-Adic Unit Ball and Communication Complexit...

Entry 346bc47abc11 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 08:55:46 UTC

Minimal Rank of p-Adic Unit Ball and Communication Complexity Rank Variance

Entry ID: 346bc47abc11 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 08:55:46 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Number Theory **Field B** (complexity object): Communication Complexity

Statement:

For any n -party communication protocol computing a function $f: \{0,1\}^n \rightarrow \{0,1\}$, the rank of the p-adic unit ball in the space of Boolean functions is linearly correlated with the variance of the matrix representing the rank of the protocols' communication complexity. Specifically, for all $n \leq 40$, $r(\text{padic_unit_ball}(f)) = \Theta(\text{Var}(\text{rank_comm_complexity}(f)))$.

Rationale (proposer's reasoning):

p-Adic units offer a unique way to encode and analyze Boolean functions due to their algebraic properties. The variance of communication complexity has been linked with the difficulty of protocols, suggesting that p-adic ranks might provide insights into this measure. This conjecture aims to bridge these two fields by proposing a quantitative relationship.

Taxonomy category: COMPLEXITY_THEORY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c1ea28dc91319929

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the correlation coefficient between the variance of the ranks of the communication matrices and the minimal ranks of the p-adic unit balls for all $n \leq 40$ is statistically significant at a 0.05 significance level using 30 seeds, with a value ≥ 0.8 . It is falsified if this correlation coefficient is < 0.5 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "p-adic number theory" AND "communication complexity" AND "rank variance" - "minimal rank of p-adic unit ball" AND communication complexity" - "arithmetic properties in Boolean functions" AND communication complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    n = len(matrix)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda x: abs(matrix[x][i]))
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        pivot = matrix[i][i]
        if pivot == 0:
            continue
        for j in range(n):
            if j != i:
                factor = Fraction(matrix[j][i], pivot)
                for k in range(n + 1):
                    matrix[j][k] -= factor * matrix[i][k]
    rank = sum(1 for row in matrix if any(row))
    return rank

def matrix_rank(matrix):
    n = len(matrix)
    m = len(matrix[0])
    A = [row[:] for row in matrix]
    rank = 0
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda x: abs(A[x][i]))
        if A[max_row][i] == 0:
            continue
        A[i], A[max_row] = A[max_row], A[i]
```

```

    pivot = A[i][i]
    for j in range(m):
        A[i][j] /= pivot
    rank += 1
    for j in range(n):
        if j != i:
            factor = A[j][i]
            for k in range(m):
                A[j][k] -= factor * A[i][k]
    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [5, 10, 15, 20, 30, 40]
    results = []

    for n in n_values:
        if n * n > 1000: # Avoid excessive memory usage
            continue

        protocol = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
        rank_comm_complexity = matrix_rank(protocol)

        p_adic_unit_ball = [sum(row[i] * (2 ** i) for i in range(n)) % n for row in protocol]
        min_p_adic_rank = min(p_adic_unit_ball)

        results.append({
            "n": n,
            "rank_comm_complexity": rank_comm_complexity,
            "min_p_adic_rank": min_p_adic_rank
        })

    if not results:
        return {
            "metric_name": "correlation_coefficient",
            "metric_value": None,
            "instances_tested": 0,
            "n_max": 0,
            "conjecture_holds": False,
            "counterexample": "no_valid_instances"
        }

    n_max = max(result["n"] for result in results)
    instances_tested = len(results)

    rank_comm_values = [result["rank_comm_complexity"] for result in results]
    min_p_adic_values = [result["min_p_adic_rank"] for result in results]

    mean_rank_comm = sum(rank_comm_values) / instances_tested
    mean_min_p_adic = sum(min_p_adic_values) / instances_tested

    variance_rank_comm = sum((x - mean_rank_comm) ** 2 for x in rank_comm_values) / instances_tested
    variance_min_p_adic = sum((x - mean_min_p_adic) ** 2 for x in min_p_adic_values) / instances_tested

    covariance = sum((rank_comm_values[i] - mean_rank_comm) * (min_p_adic_values[i] - mean_min_p_adic) for i in range(instances_tested))
    correlation_coefficient = covariance / math.sqrt(variance_rank_comm * variance_min_p_adic)

```

```

return {
    "metric_name": "correlation_coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient >= 0.8,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        std_value = math.sqrt(sum((result["metric_value"] - mean_value) ** 2 for result in results) / len(results))
        support_fraction = len([result for result in results if result["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient < 0.8\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no_valid_instances")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_88877ab4.py", line 116, in <module>
    trial_result = run_trial(seed)
                   ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_88877ab4.py", line 65, in run_trial
    rank_comm_complexity = matrix_rank(protocol)
                           ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_88877ab4.py", line 41, in matrix_rank
    if A[max_row][i] == 0:
        ~~~~~^
IndexError: list index out of range

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed before producing data, which means that the required statistical analysis to determine if the conjecture is supported or falsified could not be performed. | next: Re-run the test with proper error handling and ensure it completes successfully to provide the necessary data for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12766
2	preregistration	ollama_remote	glm4:latest	0	0	9496
3	novelty	ollama_remote	glm4:latest	0	0	16317
4	novelty	ollama_remote	glm4:latest	0	0	8721
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	52913
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11673
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10890
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14526
9	judge	ollama_remote	glm4:latest	0	0	11759

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 149061 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/346bc47abc11.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/346bc47abc11.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/346bc47abc11.tar.gz (if generated)